

EC-231 Operating Systems

BS(CE)-2021

LAB REPORT # 06



Submitted By:

Reg. No.	Name
21-CE-009	Huzaifa Shahbaz
21-CE-021	Yawar Ali
21-CE-020	Muhamamd Ibthesam

Department of Computer Engineering

BS Computer Engineering Program

HITEC University Taxila

	Presentation (Format (0.5) + Title (0.5) + Conclusion(1) + Procedure(2) + Objective(1))	Calculation/ Coding	Observation/ Program Results	Total
Total	5	5	5	15
Obtained				

Lab Instructor.
Engr. Fasih Ahmad

Experiment # 06

Implementation of File I/O Redirection System Calls in Linux

Objective

In this lab we will learn about system call techniques in Linux.

Software Used

Virtual Box

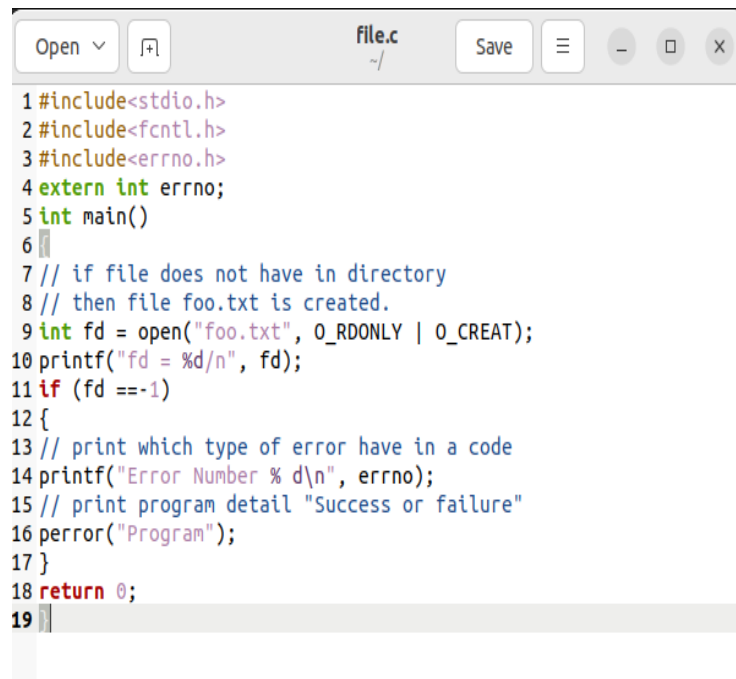
Ubuntu

Procedure

1. Generate a C file using the command ``touch``.
2. Open the file using the command ``xdg-open``.
3. Utilize the `open()` system call to establish file descriptors for both the required input and output files.
4. Employ the `dup2()` system call to replicate the file descriptors of standard input (stdin) and standard output (stdout) with those obtained previously.
5. Execute the desired command using the `execvp()` or `execv()` system call, ensuring that it reads input from the redirected input file and writes output to the redirected output file.
6. Close the file descriptors created in steps 3 and 4 using the `close()` system call to release system resources.
7. Implement error handling mechanisms to address any failures during file I/O operations or command execution, guaranteeing the robustness and reliability of the redirection process.
8. Thoroughly test the implementation with diverse input and output scenarios to validate correct redirection behavior. Verify that the desired command executes as expected with the specified input and output files.
9. Compile the C file using ``gcc file.c -o file``.
10. Display the output by executing the file using ``./file``.

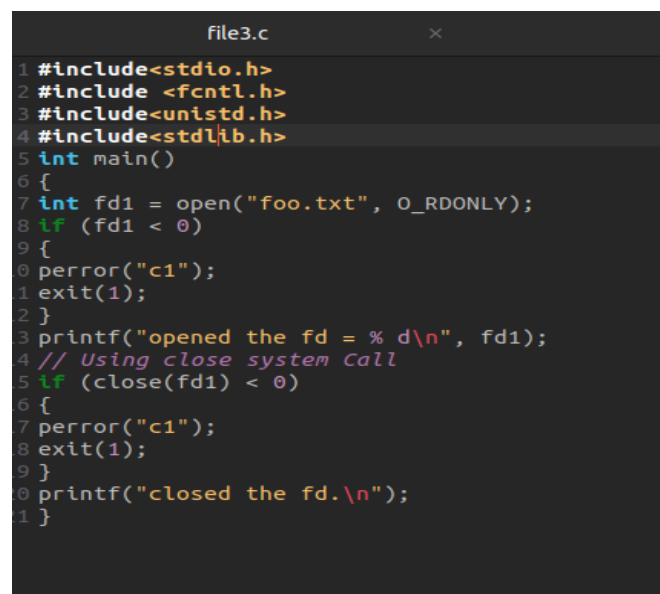
Tasks

Task 1



```
1 #include<stdio.h>
2 #include<fcntl.h>
3 #include<errno.h>
4 extern int errno;
5 int main()
6 {
7 // if file does not have in directory
8 // then file foo.txt is created.
9 int fd = open("foo.txt", O_RDONLY | O_CREAT);
10 printf("fd = %d/n", fd);
11 if (fd == -1)
12 {
13 // print which type of error have in a code
14 printf("Error Number % d\n", errno);
15 // print program detail "Success or failure"
16 perror("Program");
17 }
18 return 0;
19 }
```

```
khan@khan-VirtualBox:~$ xdg-open file.c
khan@khan-VirtualBox:~$ gcc file.c -o file10
khan@khan-VirtualBox:~$ ./file10
fd = 3/nkhan@khan-VirtualBox:~$
```



```
1 #include<stdio.h>
2 #include <fcntl.h>
3 #include<unistd.h>
4 #include<stdlib.h>
5 int main()
6 {
7 int fd1 = open("foo.txt", O_RDONLY);
8 if (fd1 < 0)
9 {
10 perror("c1");
11 exit(1);
12 }
13 printf("opened the fd = % d\n", fd1);
14 // Using close system Call
15 if (close(fd1) < 0)
16 {
17 perror("c1");
18 exit(1);
19 }
20 printf("closed the fd.\n");
21 }
```

```
khan@khan-VirtualBox:~$ gcc file.c -o file10
khan@khan-VirtualBox:~$ ./file10
opened the fd = 3
closed the fd.khan@khan-VirtualBox:~$ xdg-open file.c
```

```

1 #include<stdio.h>
2 #include<fcntl.h>
3 #include <stdlib.h>
4 #include <unistd.h>
5 int main()
6 {
7     int fd1 = open("foo.txt", O_RDONLY, 0);
8     close(fd1);
9     // assume that baz.txt is already created
10    int fd2 = open("baz.txt", O_RDONLY, 0);
11    printf("fd2 = %d\n", fd2);
12    exit(0);
13 }

```

```

khan@khan-VirtualBox:~$ gcc file.c -o file10
khan@khan-VirtualBox:~$ ./file10
fd = 3
khan@khan-VirtualBox:~$

```

```

1 #include<stdio.h>
2 #include<fcntl.h>
3 #include <stdlib.h>
4 #include<unistd.h>
5 int main()
6 {
7     int fd, sz;
8     char *c = (char *) calloc(100, sizeof(char));
9     fd = open("foo.txt", O_RDONLY);
10    if (fd < 0) { perror("r1"); exit(1); }
11    sz = read(fd, c, 10);
12    printf("called read(%d, c, 10). returned that"
13    " %d bytes were read.\n", fd, sz);
14    c[sz] = '\0';
15    printf("Those bytes are as follows: %s\n", c);
16 }

```

```

khan@khan-VirtualBox:~$ xdg-open file.c
khan@khan-VirtualBox:~$ gcc file.c -o file10
khan@khan-VirtualBox:~$ ./file10
called read( 3, c, 10). returned that 0 bytes were read

```

```

1 #include<stdio.h>
2 #include<fcntl.h>
3 #include <stdlib.h>
4 #include<unistd.h>
5 int main()
6 {
7     char c;
8     int fd1 = open("sample.txt", O_RDONLY, 0);
9     int fd2 = open("sample.txt", O_RDONLY, 0);
10    read(fd1, &c, 1);
11    read(fd2, &c, 1);
12    printf("c = %c\n", c);
13    exit(0);
14 }

```

```

khan@khan-VirtualBox:~$ xdg-open file.c
khan@khan-VirtualBox:~$ gcc file.c -o file10
khan@khan-VirtualBox:~$ ./file10
c = f
khan@khan-VirtualBox:~$

```

```

1 #include<stdio.h>
2 #include<fcntl.h>
3 #include<stdlib.h>
4 #include<unistd.h>
5 #include<string.h>
6 int main (void)
7 {
8     int fd[2];
9     char buf1[12] = "hello world";
10    char buf2[12];
11    // assume foobar.txt is already created
12    fd[0] = open("foobar.txt", O_RDWR);
13    fd[1] = open("foobar.txt", O_RDWR);
14    write(fd[0], buf1, strlen(buf1));
15    write(1, buf2, read(fd[1], buf2, 12));
16    close(fd[0]);
17    close(fd[1]);
18    return 0;
19 }
20

```

```

khan@khan-VirtualBox:~$ xdg-open file.c
khan@khan-VirtualBox:~$ xdg-open file.c
khan@khan-VirtualBox:~$ gcc file.c -o file10
khan@khan-VirtualBox:~$ ./file10
hello world
khan@khan-VirtualBox:~$

```

```

#include<stdio.h>
#include<fcntl.h>
#include<unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
int main()
{
    int fd; char
    buffer[80];
    static char message[] = "Hello, world";
    fd = open("created_file.txt",O_RDWR|O_CREAT);
    printf("fd = %d \n", fd);
    if (fd != -1)
    {
        printf("myfile opened for read/writeaccess\n");
        write(fd, message, sizeof(message));
        lseek(fd, 0, 0); /* go back to the beginning of the file */
        read(fd, buffer, sizeof(message));
        printf(" %s was written to myfile \n",buffer);
        close (fd);
    }
}

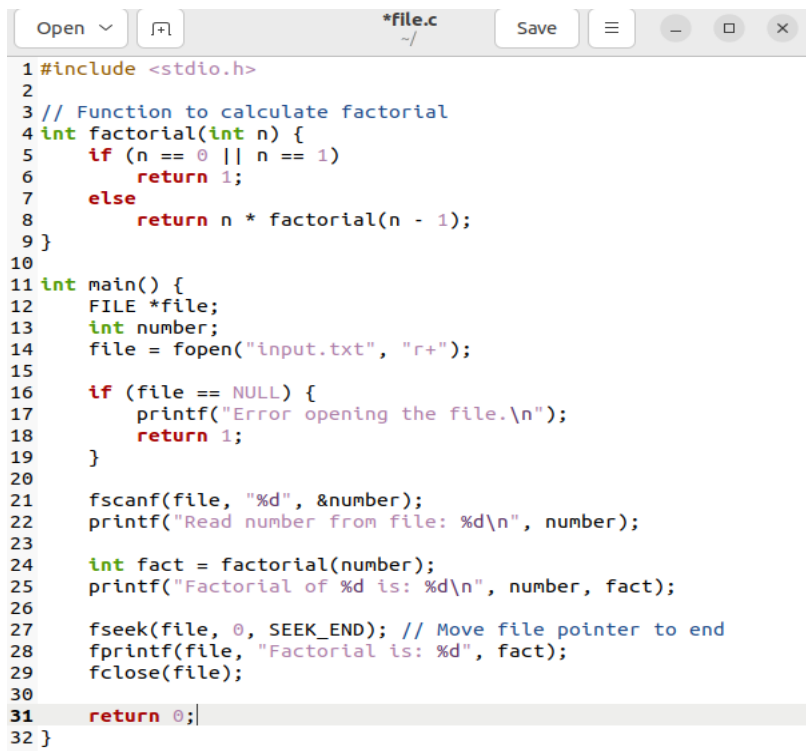
```

```

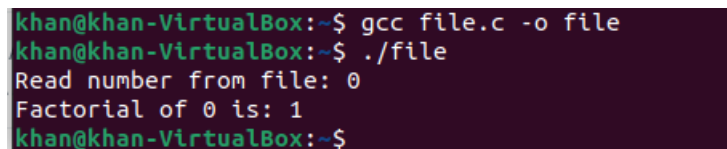
khan@khan-VirtualBox:~$ xdg-open file.c
khan@khan-VirtualBox:~$ gcc file.c -o file10
khan@khan-VirtualBox:~$ ./file10
Fd = 3
myfile opened for read/writeaccess
Hello, world was written to myfile
khan@khan-VirtualBox:~$

```

Task 2



```
1 #include <stdio.h>
2
3 // Function to calculate factorial
4 int factorial(int n) {
5     if (n == 0 || n == 1)
6         return 1;
7     else
8         return n * factorial(n - 1);
9 }
10
11 int main() {
12     FILE *file;
13     int number;
14     file = fopen("input.txt", "r+");
15
16     if (file == NULL) {
17         printf("Error opening the file.\n");
18         return 1;
19     }
20
21     fscanf(file, "%d", &number);
22     printf("Read number from file: %d\n", number);
23
24     int fact = factorial(number);
25     printf("Factorial of %d is: %d\n", number, fact);
26
27     fseek(file, 0, SEEK_END); // Move file pointer to end
28     fprintf(file, "Factorial is: %d", fact);
29     fclose(file);
30
31     return 0;
32 }
```



```
khan@khan-VirtualBox:~$ gcc file.c -o file
khan@khan-VirtualBox:~$ ./file
Read number from file: 0
Factorial of 0 is: 1
khan@khan-VirtualBox:~$
```

Conclusion

In conclusion, leveraging File I/O redirection system calls in Linux facilitates effective management of file content within applications. Through the utilization of functions such as open, read, write, and close, programmers can efficiently execute file operations. By integrating error handling mechanisms, software robustness is enhanced, while following established file handling best practices fosters reliability and ease of maintenance.